

函數 or 函式

Python: Basic Functions

Cheng-Te Li (李政德)

chengte@mail.ncku.edu.tw

Dept. of Statistics, NCKU



Course Info (12/17)

- Let me know if you have any concern on midterm
- No worry about your score (except for the last ones)
- **Programming exam 3** will be at **12/31**
- **Final exam** will be at **1/7**
 - Hint: be familiar with midterm and HWs
- **Good news for you 😊**
 - Select 2 highest from 3 programming exams
 - Select 4 highest from 5 HWs (HW1-HW5)
- **HW4** deadline: **12/23** 23:59 (Mon)
- **HW5** is out, Deadline: **1/10** 23:59 (Fri)
 - Topic: **Movie Data Analysis**
 - String, Loops, List, Function, Files, Dictionary
 - Whether to do it, it's up to you

Functions

So far, your Python program was one continuous list of instructions



```
# 1. assign values to variables
a = 10
b = 5
# 2. calculate sum of variables
c = a + b
# 3. print resulted sum
print(c)
# 4. repeat steps 1-3
b = 0.5
c = a + b
print(c)
```

2019/12/17

You can simplify this by dividing the whole work into several smaller tasks, which each becomes a function



```
def sumNum(a, b):
    c = a + b
    return c

# main code: function calls
print(sumNum(10, 5))
print(sumNum(10, 0.5))
```

Functions

- Mathematics

- Functions are (typically named) formulae describing how some quantity (result) may be calculated from one or more “inputs”(arguments)

$$f(x) = 5x^2 + 12x - 13$$

- Python

- A named block of code or “subprogram”
- Typically:
 - Take in some “inputs” arguments
 - Calculate a result
 - Code block describes steps involved
- “Subprogram” may be called (executed) from various points in the code

Functions

- Purposes
 - Code reusability
 - Your programs become much more readable
 - Better code organization
 - Less errors – if similar functionalities appear more than once in your program, just create and invoke a function many times (better to test, less bugs, shorter code)
- Functions are programmer created code blocks that do a specific task
- Functions are same as black boxes
 - Once a function is defined, one can use it without knowing how it was implemented (the code inside)
 - Functions encapsulate details

Some Familiar Functions

- Some Python functions we have seen already
 - `len("Python")` # the length of the string
 - `print("hello world!")` # print out a string
 - `"hello".replace("l", "k")` # replace strings

- Syntax:

`biggest = max(x, y, z)`



Function name Arguments

- Apart from these built-in functions,
we can write our own

max () Function

- A function is some stored code that we use
- A function takes some **input** and produces an **output**

```
x, y, z = 8, 3, 5  
big = max(x, y, z)  
print(big)      # 8
```



Guido wrote this code

```
def max(a, b, c):  
    largest = c  
    if a > b and a > c:  
        largest = a  
    elif b > a and b > c:  
        largest = b  
    return largest
```

Input →

x, y, z
8 3 5

→ **Output**

8

Define Your Own Function



```
def func_name(parameter_1, ..., parameter_n):  
    """A docstring describing the function."""  
    statements  
    ...  
    return result
```

- Convention for **function names** and formal parameters: lowercase with underscore
- Docstring, **parameters**, and **return** are **optional**
- **return can occur anywhere** in the function
 - **Terminate the function and return the return value**
A function with no return statement returns `None` once if there are no more statements to execute

Define Your Own Function

- Start a function definition with the keyword **def**
- Then comes the **function name**, with **parameters** in braces, and then a **colon**

```
def func(par1, par2, ...) :
```

- Then comes, **indented**, the body of the function
- Use **return** to specify the output

```
def calc_circumference(radius):  
    pi = 3.14  
    circumference = 2 * pi * radius  
    return circumference
```

Define Your Own Function

- This defines the function **but does not execute the body of the function**

```
x = 5  
print("Hello")
```

```
def print_lyrics():  
    print("Jingle bell Jingle bell Jingle all the way~")  
    print('Wish you a merry Christmas and a happy new year!')
```

```
print("YO!")  
x = x + 5  
print(x)
```

Hello
Yo
10

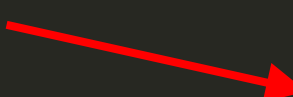
Define Your Own Function

- Once we have **defined** a function, we can **call** (or **invoke**) it as many times as we like
- This is the **store** and **reuse** pattern

```
x = 5  
print("Hello")
```

```
def print_lyrics():  
    print("Jingle bell Jingle bell Jingle all the way~")  
    print('Wish you a merry Christmas and a happy new year!')
```

```
print("YO!")  
print_lyrics()  
x = x + 5  
print(x)
```



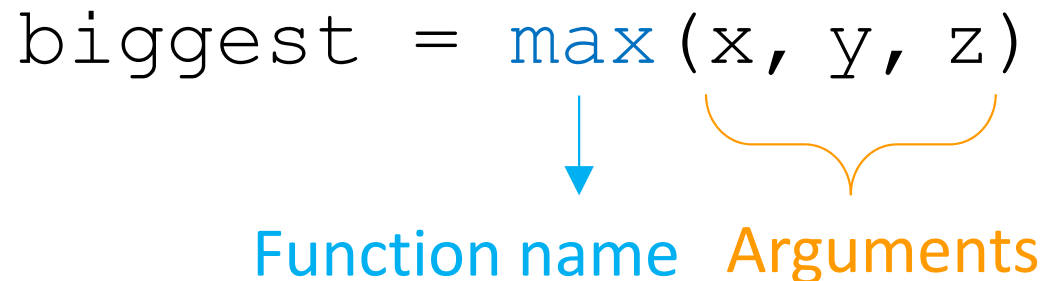
```
Hello  
Yo  
Jingle bell Jingle bell Jingle all the way~  
Wish you a merry Christmas and a happy  
new year!
```

```
10
```

Arguments

- An **argument** is a value we pass into the **function** as its **input** when we call the function
- We use **arguments** so we can direct the **function** to do different kinds of work when we call it at **different times**
- We put the **arguments** in parentheses after the name of the **function**

```
biggest = max(x, y, z)
```



Function name Arguments

Parameters

- A **parameter** is a variable **used in the function definition**
- A parameter is a “handle” that allows the code in the function to **access the arguments** for a particular function invocation

```
def great(lang):  
    if lang == "es":  
        print("Hola")  
    elif lang == "fr":  
        print("Bonjour")  
    else:  
        print("Hello")
```

```
great("en")  
great("es")  
great("fr")
```



```
Hello  
Hola  
Bonjour
```

Multiple Parameters / Arguments

- We can define **more than one parameter** (e.g. 2, 3, 4, ..., parameters) in the function definition
- Simply **add more arguments** when we call the function
- **Need to match the number and order of arguments and parameters**

```
def addtwo(a, b):  
    # Return the summation of two values  
    added = a + b  
    return added  
  
x = addtwo(3, 5)  
print(x)
```

Every function should have header comment describing succinctly what it does

Return Values

- Often a function will take its arguments, do some computation, and **return a value to be used as the value of the function call** in the calling expression
- The **return** statement ends the function execution and “sends back” the result of the function

```
def great():  
    return "Hello"  
print(great(), "Mary")  
print(great(), "John")
```



Hello Mary
Hello John

Hello Mary
Hola John
Bonjour Tom



```
def great(lang):  
    if lang == "es":  
        return "Hola"  
    elif lang == "fr":  
        return "Bonjour"  
    else:  
        return "Hello"  
print(great("en"), "Mary")  
print(great("es"), "John")  
print(great("fr"), "Tom")
```

Function Terminology

- Definitions

- ▣ **Function header**: specify name and parameters (in parentheses)
- ▣ **Function body**: indented block of statements; specify actions to be taken when function is executed
- ▣ **One definition, possibly many calls**

- Parameter/Argument

- ▣ **Parameter**: e.g. `n`: function variable; specify "input" to function; function body expressed in terms of these parameters
- ▣ **Argument**: e.g. `num=17` actual value supplied in function call; used to initialize parameter

```
def square(n):  
    return n ** 2 # return the square of n  
num = int(input("Please enter an integer"))  
result = square(num) # this is a function call  
print(result)
```

Function Execution

- 1) Variable `num` assigned value typed in by user (say 7)
- 2) Evaluation of RHS of assignment begins; function `square` "called" with argument 7
- 3) Function `square` starts; ("main" program "sleeps" until `square` returns); argument value 7 used to initialize parameter `n`
- 4) Expression `n**2` evaluated (result 49); `square` finishes and returns value 49
- 5) "main" program reawakens and assigns 49 to `result`
- 6) The value of `result` (49) printed

```
def square(n):                                     # (3)
    return n ** 2                                  # (4)
num = int(input("Please enter an integer"))         # (1)
result = square(num)                               # (2): call, (5): assignment
print(result)                                     # (6)
```

Example: Temperature Conversion Table

```
def celsius_to_fahr(celsius):  
    # Return Fahrenheit equivalent to given temperature  
    fahr = celsius * 9 / 5 + 32  
    return fahr  
  
for temp in range(-10, 50, 5):  
    print(temp, "C = ", celsius_to_fahr(temp), "F")
```

```
c:\Python35-32\workspace>python func_temp.py  
-10 C = 14.0 F  
-5 C = 23.0 F  
0 C = 32.0 F  
5 C = 41.0 F  
10 C = 50.0 F  
15 C = 59.0 F  
20 C = 68.0 F  
25 C = 77.0 F  
30 C = 86.0 F  
35 C = 95.0 F  
40 C = 104.0 F  
45 C = 113.0 F
```

Program Structure

```
# Define function at the beginning
def function1(...):
    """ write what to do in this function """
    # body_of_function1
def function2(...):
    """ write what to do in this function """
    # body_of_function2
# ... other function definitions
# Main program code which includes function calls
statements          # usually input from uses
variable assignment  # values assigned to variables
function1(...)       # call of function1
function2(...)       # call of function1
# ... other statements
```

To function or not to function...

- Organize your code into “paragraphs”
→ capture a complete thought and “name it”
- Don't repeat yourself
→ make it work once and then reuse it
- If something gets too long or complex,
→ break it up into logical chunks and put those chunks in functions
- Make a library of common stuff that you do over and over –
perhaps share this with your friends...

Example: Create a Deck and Draw Some Cards

```
1 import random
2 ranks = ['A', '2', '3', '4', '5', '6', '7',
3          '8', '9', '10', 'J', 'Q', 'K']
4 suits = ['C', 'D', "H", 'S']
5
6 def createShuffledDeck():
7     # make a deck of cards and shuffle it
8     deck = []
9     for s in suits:
10         for r in ranks:
11             deck.append(s+r)
12     random.shuffle(deck)
13     return deck
14
15 def drawCard(deck, num):
16     # draw num cards at random
17     cards = []
18     for i in range(num):
19         card = deck.pop(0)
20         cards.append(card)
21     return cards
22
23 deck = createShuffledDeck()
24 print(drawCard(deck, 1,), " #cards=", len(deck))
25 print(drawCard(deck, 3,), " #cards=", len(deck))
26 print(drawCard(deck, 2,), " #cards=", len(deck))
```

Output:

```
['S8'] #cards= 51
['H5', 'D6', 'H2'] #cards= 48
['H8', 'S10'] #cards= 46
```

In-class Exercise: BMI and Heart Disease Risk

$$\text{BMI} = \frac{\text{weight (kg)}}{\text{height}^2 \text{ (m}^2\text{)}}$$

```
1 tom_h, tom_w, tom_a = 1.8, 120, 50
2 ada_h, ada_w, ada_a = 1.6, 40, 20
3
4 def compute_BMI(height, weight):
5     # compute the BMI value for the given height and weight
6     bmi = weight / (height ** 2)
7     return bmi
8
9 def HeartRisk(bmi, age):
10    # get the risk level of heart disease for the input BMI
11    table = [['medium', 'high'], ['low', 'medium']]
12    young = age < 45
13    heavy = bmi >= 22.0
14    risk = table[young][heavy]
15    return risk
16
17 tom_bmi = compute_BMI(tom_h, tom_w)
18 print("Tom: BMI = %.2f, Risk = %s" % (tom_bmi, HeartRisk(tom_bmi, tom_a)))
19 ada_bmi = compute_BMI(ada_h, ada_w)
20 print("Ada: BMI = %.2f, Risk = %s" % (ada_bmi, HeartRisk(ada_bmi, ada_a)))
```

BMI	Age	
	< 45	≥ 45
< 22	Low	Medium
≥ 22	Medium	High